Project 2: Detecting Anomalies in Credit Card Transactions

CUSUM Algorithm — Implementation, Experiments and Results

This report describes a Python implementation of a two-sided CUSUM detector for the supplied 100,000-row credit-card transaction dataset. The implementation follows the assignment brief: read a transaction stream, update positive and negative cumulative sums, flag threshold crossings, and report evaluation statistics.

1. Dataset description

The CSV contains 100,000 records and 50 distinct customers. It includes transaction ID, timestamp, customer ID, merchant category, country, amount, and a reference label named *is_anomaly*. The reference labels contain 99,000 normal transactions and 1,000 anomaly-labelled transactions. The mean amount is 52.53, the median is 50.19, and the standard deviation is 27.36.

The label column is reserved for evaluation only. It is not used to estimate customer baselines or to make CUSUM predictions, avoiding target leakage.

2. CUSUM principle

CUSUM (Cumulative Sum Control Chart) monitors cumulative deviations from an expected reference value. The assignment brief defines two sums: S+ for positive shifts and S– for negative shifts. When either sum becomes greater than the decision threshold h, the process is considered to have shifted and an anomaly alert

For this dataset, each customer is calibrated independently. The first 100 transactions for each customer provide a reference mean (μ_0) and standard deviation (σ_0). Later transactions are standardized as $z_t = (x_t - \mu_0) / \sigma_0$. The detector then applies the brief's two-sided recurrence with $\mu_0 = 0$ in standardized units: $S^+_t = \max(0, S^+_{t-1} + z_t - k)$, and $S^-_t = \max(0, S^-_{t-1} - z_t - k)$.

3. Implementation and parameter selection

The input is sorted by timestamp and transaction ID. The program maintains an independent state for each customer, so one customer's spending pattern does not affect another customer's CUSUM state. The first 100 rows per customer are calibration rows and are not scored. The selected parameters are k = 4.0 and h = 15.0 in standardized units. These values were selected as a demonstration trade-off on the supplied data: the tolerance reduces accumulation from ordinary variation, while the threshold determines how much cumulative evidence is required before an alert.

A smaller k or h makes the detector more sensitive and usually increases the number of alerts and false positives. A larger k or h is more conservative and may miss genuine anomalies. The best values depend on the cost of false alarms versus missed fraud and should be selected on a separate validation period in a production setting.

4. Results

After excluding calibration rows, the evaluation set contains 95,000 transactions, including 951 reference anomaly labels. CUSUM flagged 903 transactions.

Metric	Result
True positives	366
True negatives	93,512
False positives	537
False negatives	585
Precision	0.405
Recall	0.385
F1-score	0.395

Accuracy	0.988

The results show that CUSUM identifies a portion of the reference anomalies, but also produces false positives and misses some labelled anomalies. This is expected because CUSUM detects accumulating shifts rather than classifying every isolated transaction independently. The result should therefore be interpreted as an anomaly-monitoring demonstration, not a production fraud decision system.

5. Visual analysis

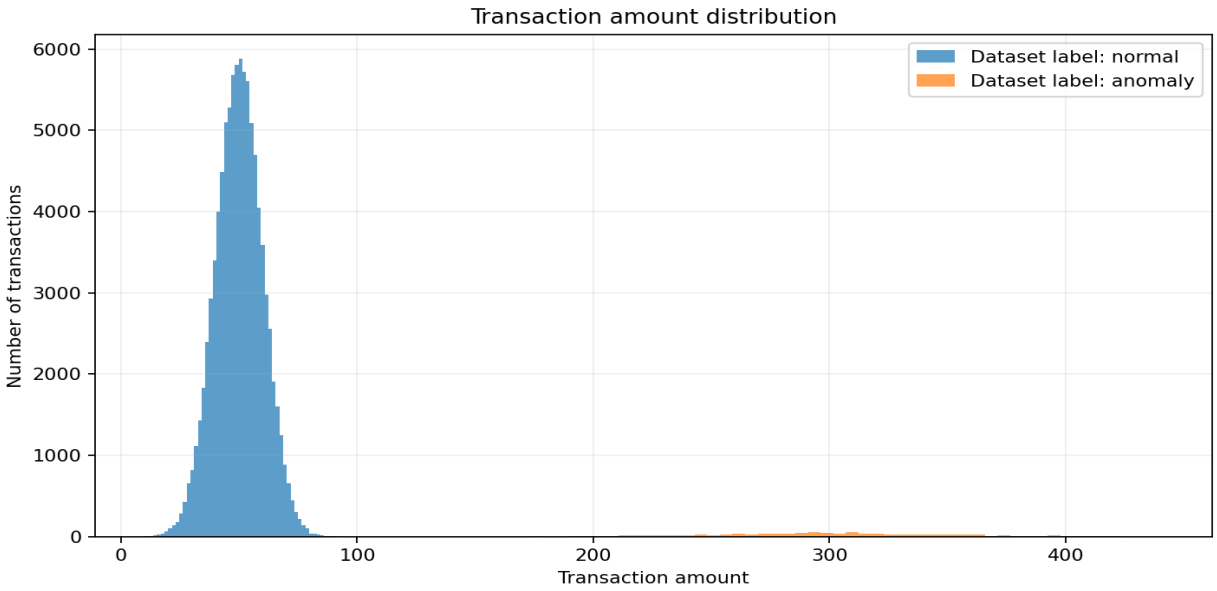


Figure 1. Transaction amount distributions by reference label.

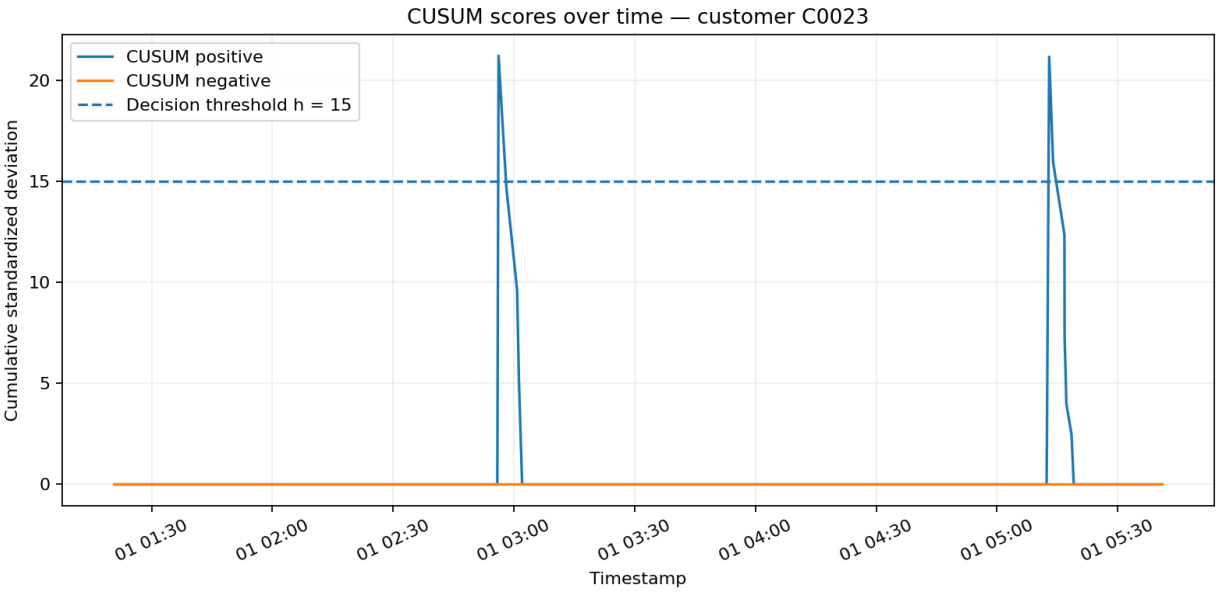


Figure 2. Positive and negative CUSUM scores over time for one customer; the dashed line is h = 15.

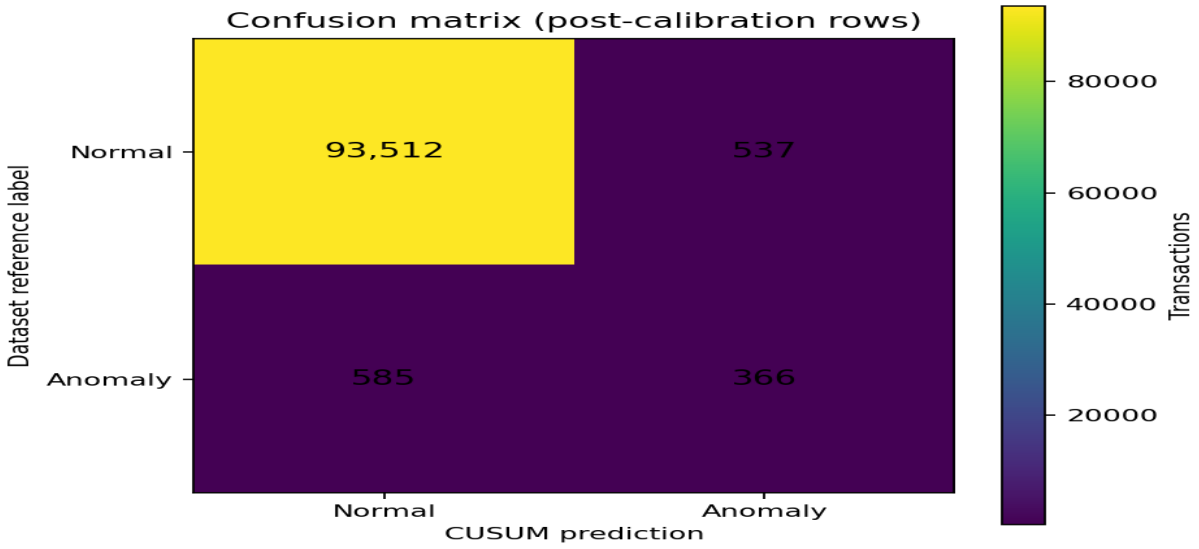


Figure 3. Confusion matrix on post-calibration transactions.

6. Conclusion

The project demonstrates how a two-sided CUSUM algorithm can monitor a transaction stream without retaining or repeatedly analysing the full historical dataset. Only a small state (reference mean, standard deviation, positive sum, negative sum, and count) is maintained for each customer. The supplied dataset shows that CUSUM can detect cumulative deviations, but performance depends strongly on calibration quality and parameter values. Further work could include validation-based parameter tuning, category-specific baselines, and a review queue for human investigation.

7. Reproducibility

Install the packages listed in requirements.txt, then run: `python src/cusum_detector.py --input data/transactions_100k_clean.csv --output-dir results`. Run the automated tests with `python -m unittest discover -s tests -v`.